
MASIBULELE NKOSI
ST10490439
PROG5121POE
PROGRAMMING 1A PART 2 (POE)

# Table of Contents


.....1

**Table of Contents**..... **Error! Bookmark not defined.**

**1. Project Overview** .....3

**Technology stack** .....3

**Architecture** .....3

**2. MVC Application Setup**.....3

**Project structure** .....3

**Consuming the API**.....4

**3. Authentication and Role-Based Navigation**.....4

**4. Pages by Role** .....6

**5. Organiser Interface** .....6

**6. Participant Interface** .....10

**7. Azure Blob Storage** .....12

**8. Docker** .....13

**9. User Interface Design** .....15

**10. GitHub and CI/CD**.....16

**11. Setup and Run Instructions** .....16

**12. Deviations from the Plan**.....16

**References** .....17

# 1. Project Overview

RaceDay is a web-based event management system for South African road running, walking and cycling events. Part 1 planned the system, Part 2 built the RESTful API, and Part 3 (this document) builds the ASP.NET Core MVC web application that consumes that API, integrates Azure Blob Storage for image uploads, and containerises the web application with Docker.

The MVC application has no direct database access and no duplicated business logic. Every piece of data is read and written through the Part 2 API, and all file uploads go through the API, which stores them in Azure Blob Storage. The MVC application never communicates with Azure directly.

## Technology stack

| Area | Technology |
|---|---|
| Web application | ASP.NET Core MVC (Models, Views, Controllers), .NET 10, C# |
| API consumption | HttpClient through IHttpClientFactory (typed clients ApiClient and UploadClient) |
| Authentication | Session-based; the API session is kept for each user and the role is stored in the MVC session |
| File storage | Azure Blob Storage, accessed only by the API (Azure.Storage.Blobs) |
| User interface | Bootstrap 5 with a custom RaceDay theme (site.css) |
| Containerisation | Docker and Docker Compose |
| CI/CD | GitHub Actions (builds the API, tests and MVC project, runs unit tests) |

## Architecture

```
Browser  -->  RaceDay.Web (MVC)  -->  RaceDay.Api (Web API)  -->  SQL Server
                                                |
                                                +-->  Azure Blob Storage (images)
```

When a user logs in, the MVC application calls the API login endpoint. The API creates its own session and returns a session cookie. The MVC application saves that cookie, together with the user's id, name and role, in its own server-side session. For every later request the MVC application forwards the saved cookie to the API, so the API can enforce the role rules on every call. This keeps the authenticated state and role across all pages.

# 2. MVC Application Setup

The RaceDay.Web project was created inside the same GitHub repository as the API. It is configured with an ApiBaseUrl setting in appsettings.json, session support, and two typed HTTP clients.

## Project structure

```
RaceDay/
├── RaceDay.Api/                Part 2 API (plus the upload endpoint for Part 3)
├── RaceDay.Api.Tests/          Unit tests
├── RaceDay.Web/
│   ├── Controllers/            Home, Account, Organiser, Participant, Events, Profile
│   ├── Filters/                RequireLoginAttribute (login and role checks)
│   ├── Models/                 API models and view models
│   ├── Services/               ApiClient, UploadClient
│   ├── Views/                  Razor views for every page
│   ├── wwwroot/css/site.css    RaceDay theme
│   ├── Dockerfile
│   └── Program.cs
├── docker-compose.yml
└── .github/workflows/ci.yml
```

### Consuming the API

- ApiClient sends every request to the API, attaches the saved session cookie, captures the session cookie returned by the login call, and converts the response into a result object.
- UploadClient sends image files to the API upload endpoint as multipart form data.
- If the API returns an error or is unavailable, the clients convert it into a friendly message so that users never see raw error messages or crashes.
- When the API reports that the session has expired (401), the user is signed out and redirected to the login page.

## 3. Authentication and Role-Based Navigation

- Users register and log in through the MVC pages, which call the API register and login endpoints.
- The user's id, name and role are stored in the MVC session, so the state is kept across all pages.
- The RequireLogin filter protects controllers. A user who is not logged in is redirected to the login page, and a user with the wrong role is sent to an Access Denied page.
- The navigation menu changes with the role: Organisers see Dashboard and Manage Events; Participants see My Enrolments and My Results; both see Profile.
- Logging out clears the MVC session and the API session, then returns the user to the home page.

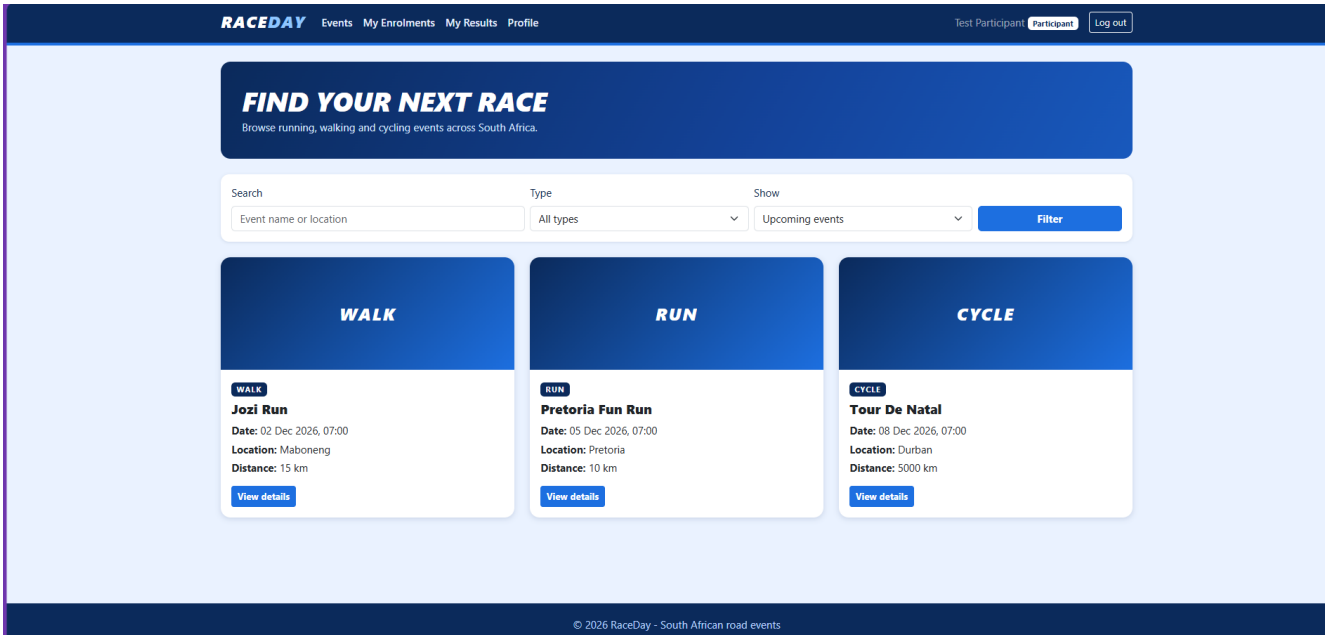



*Figure 1: Home page for a visitor who is not logged in (only Log in and Register are shown)*

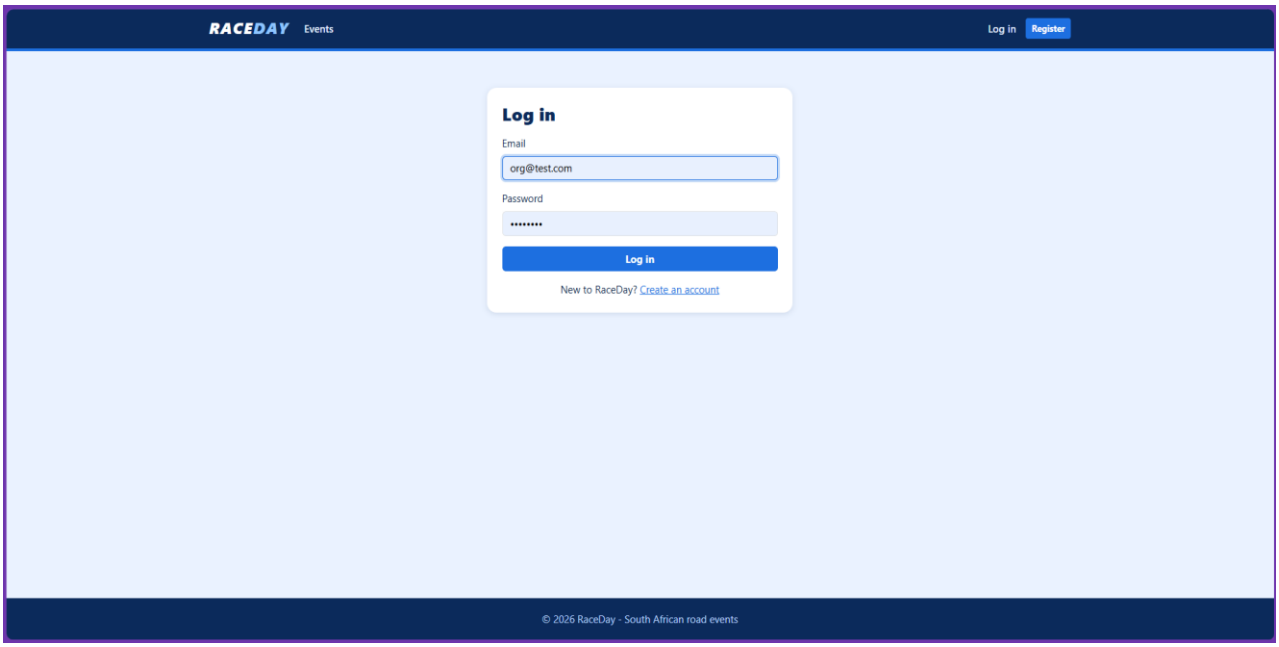



*Figure 2: Login page*

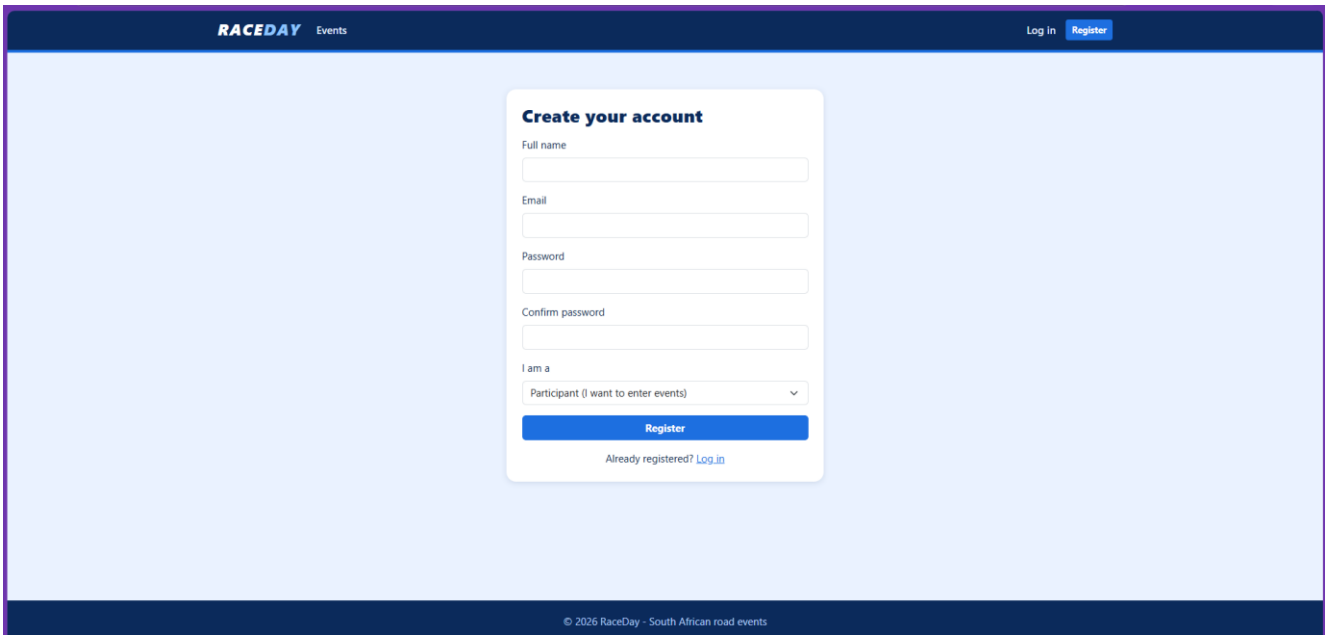



*Figure 3: Register page with the role selection*

## 4. Pages by Role

| Page | Who can use it | Purpose |
|---|---|---|
| Home (Events) | Public | Lists upcoming events with search, type and date filters; each card links to the event details. |
| Event details | Public | Shows the full event information and its categories. A logged-in Participant can enrol by choosing a category. |
| Login / Register | Public | Creates an account (choosing Organiser or Participant) and starts a session by calling the API. |
| Dashboard | Organiser | Summary of the Organiser's events, upcoming events and total enrolments per event. |
| Manage Events | Organiser | Create, edit and delete events, with an optional banner image upload. |
| Categories | Organiser | Add age or distance categories to an event. |
| Enrolments | Organiser | Lists every Participant enrolled in an event, with their category and status. |
| Results | Organiser | Captures the finish time and finishing position for each enrolled Participant. |
| My Enrolments | Participant | Shows the events the Participant entered, with colour-coded status. |
| My Results | Participant | Shows event, date, category, finish time and position (for example 47th out of 312 finishers). |
| Profile | Organiser and Participant | View and update the name and upload a profile picture. |

## 5. Organiser Interface

The Organiser interface is only available to users with the Organiser role. The Organiser can only manage events they created; the API checks ownership on every action.

- Dashboard: total events, upcoming events, total enrolments and a table of events with their enrolment counts.
- Manage Events: create, edit and delete events with name, description, date, location, distance and event type, and an optional banner image.
- Categories: add age or distance categories to an event.
- Enrolments: view every Participant enrolled in an event, with their selected category and status.
- Results: enter the finish time and finishing position for each enrolled Participant after the event.

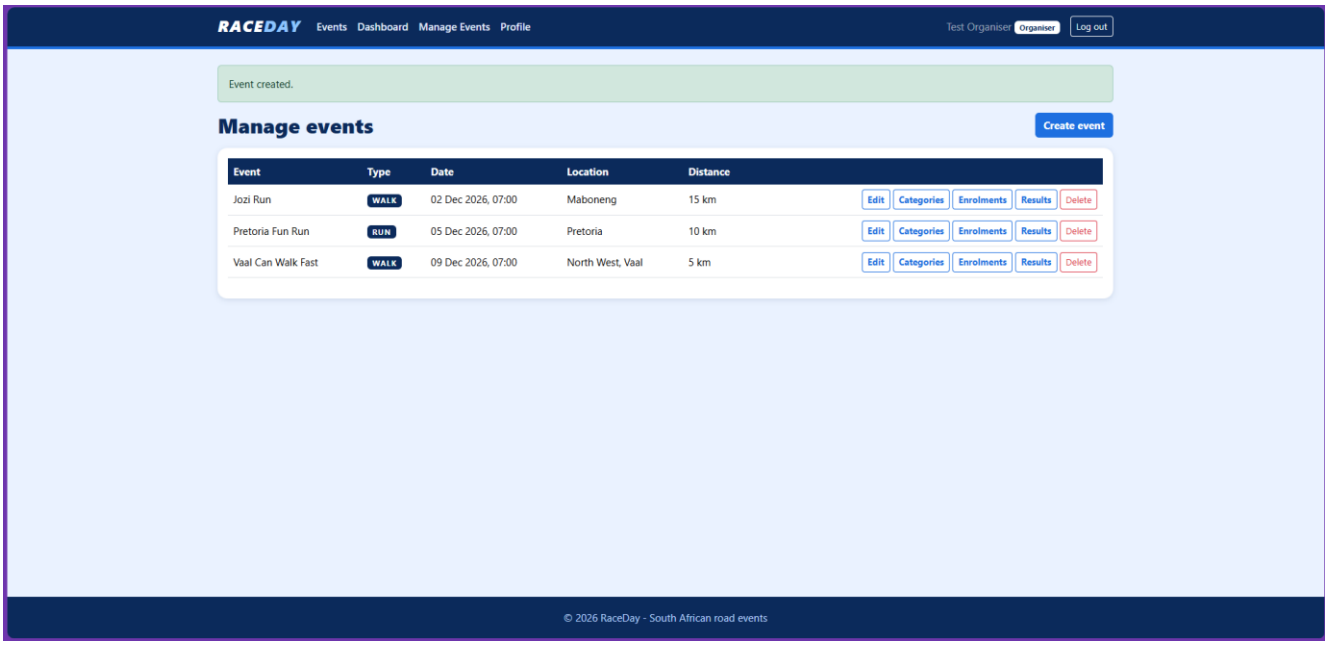



*Figure 4: Manage events page for the Organiser (Edit, Categories, Enrolments, Results and Delete)*

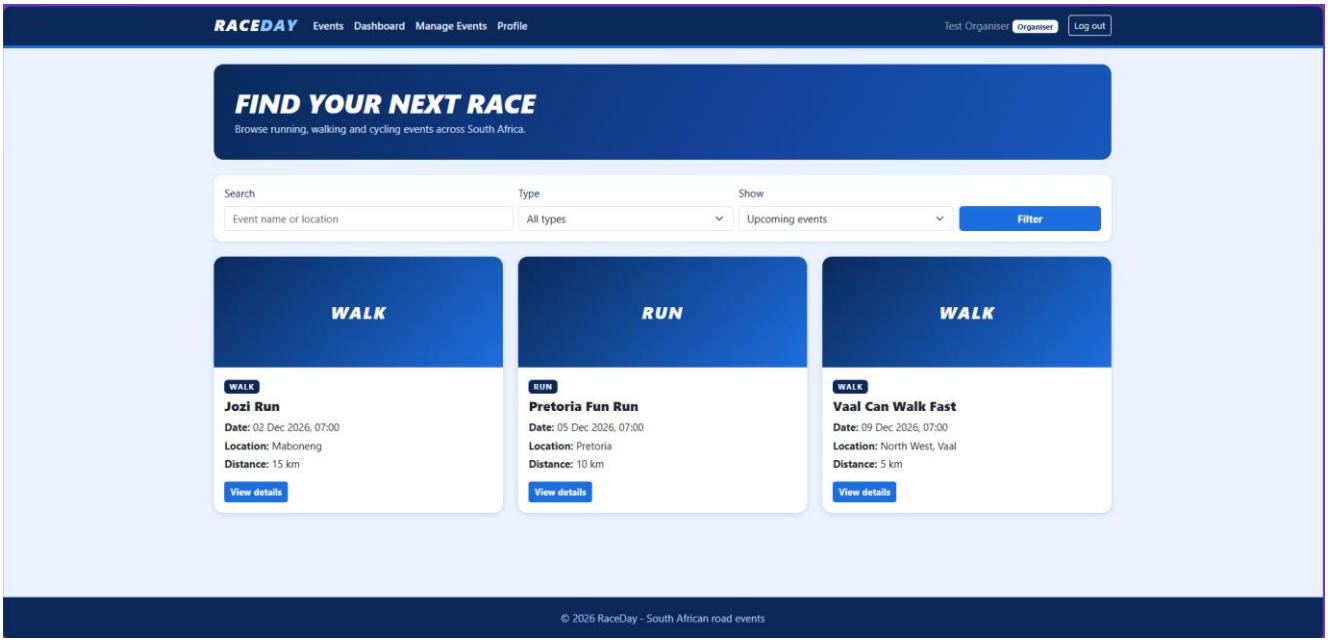



*Figure 5: The Organiser's navigation menu (Dashboard, Manage Events and Profile) on the events page*

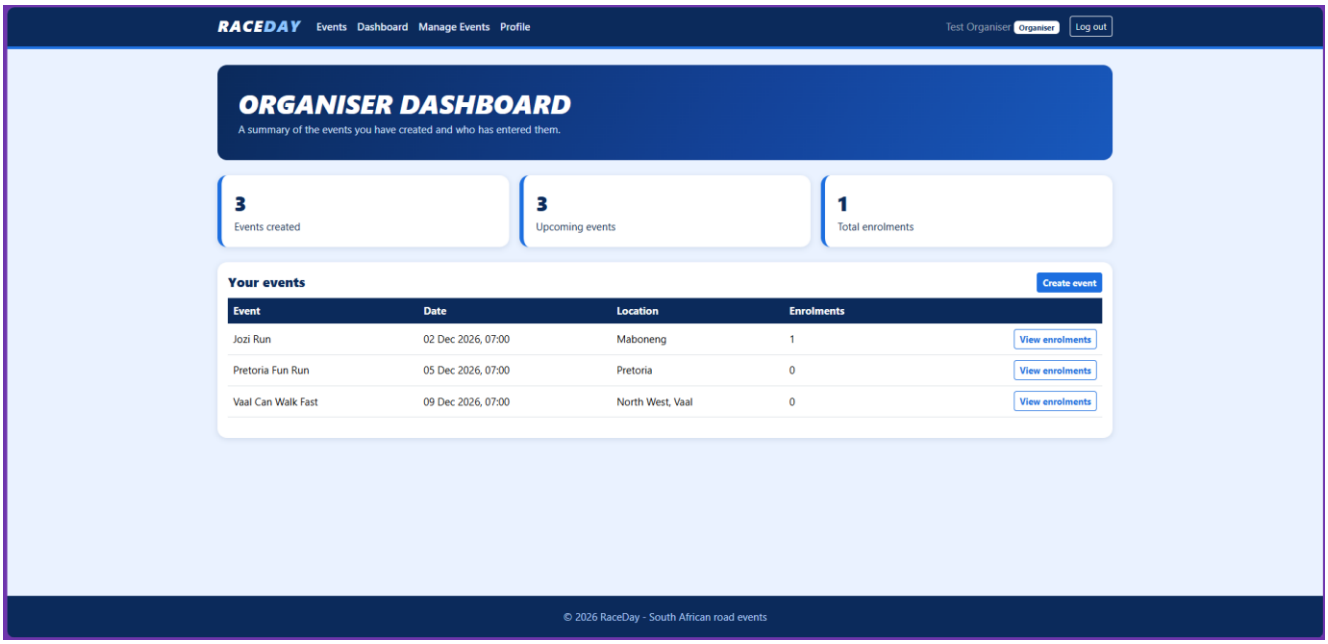



*Figure 6: Organiser dashboard*

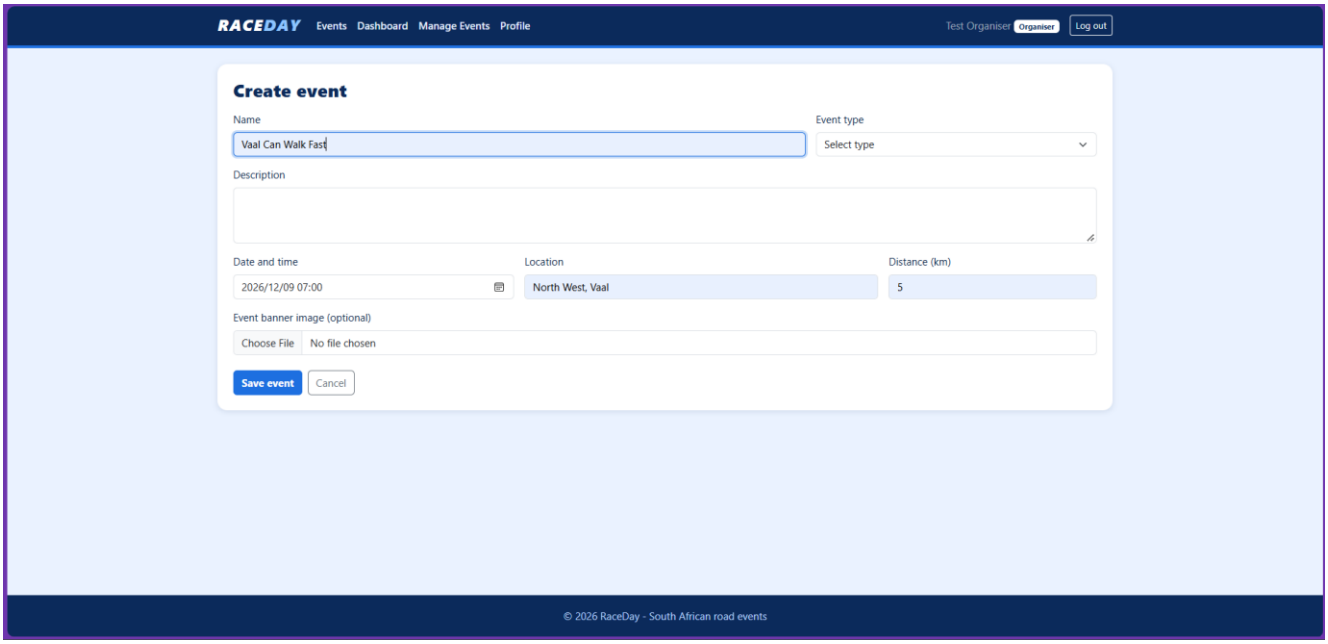



*Figure 7: Create or edit event form (with banner image upload)*

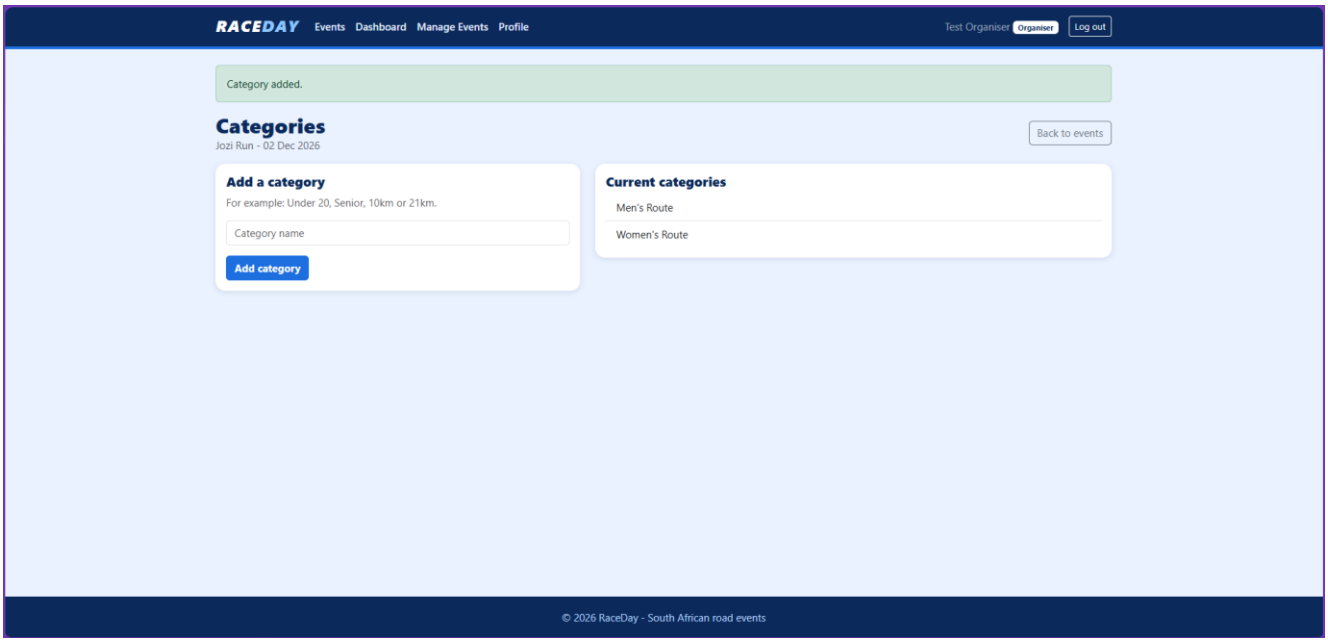



*Figure 8: Categories page*

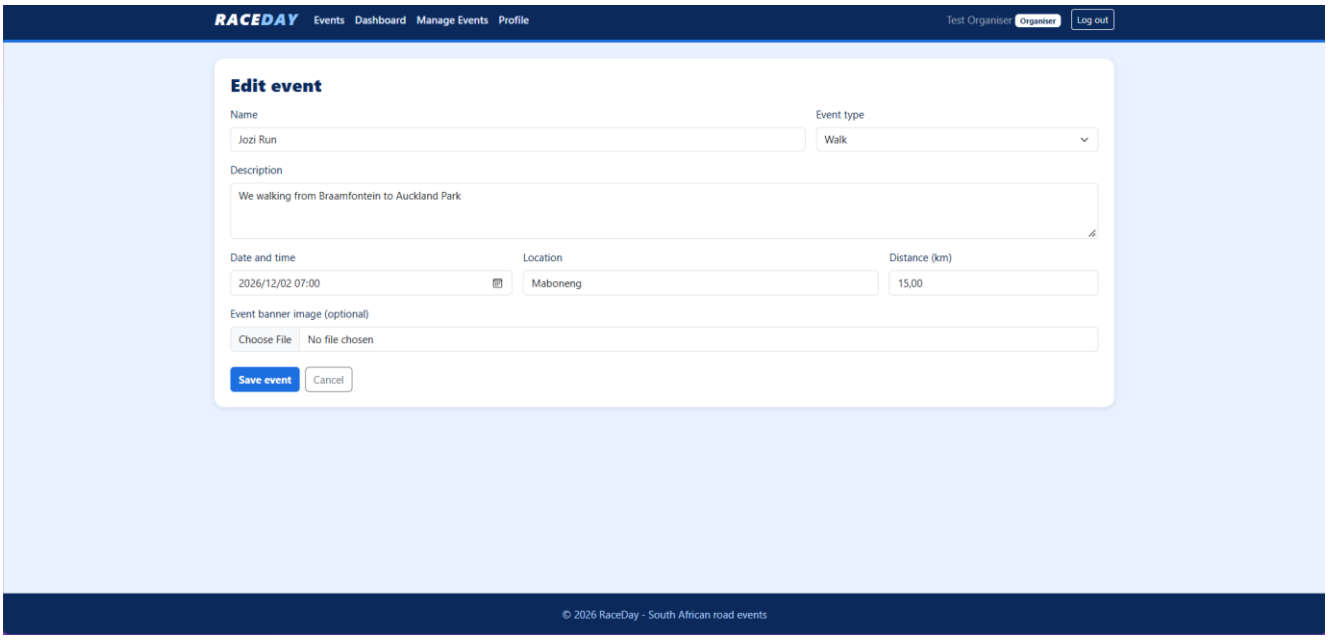



*Figure 9: Enrolments page for an event*

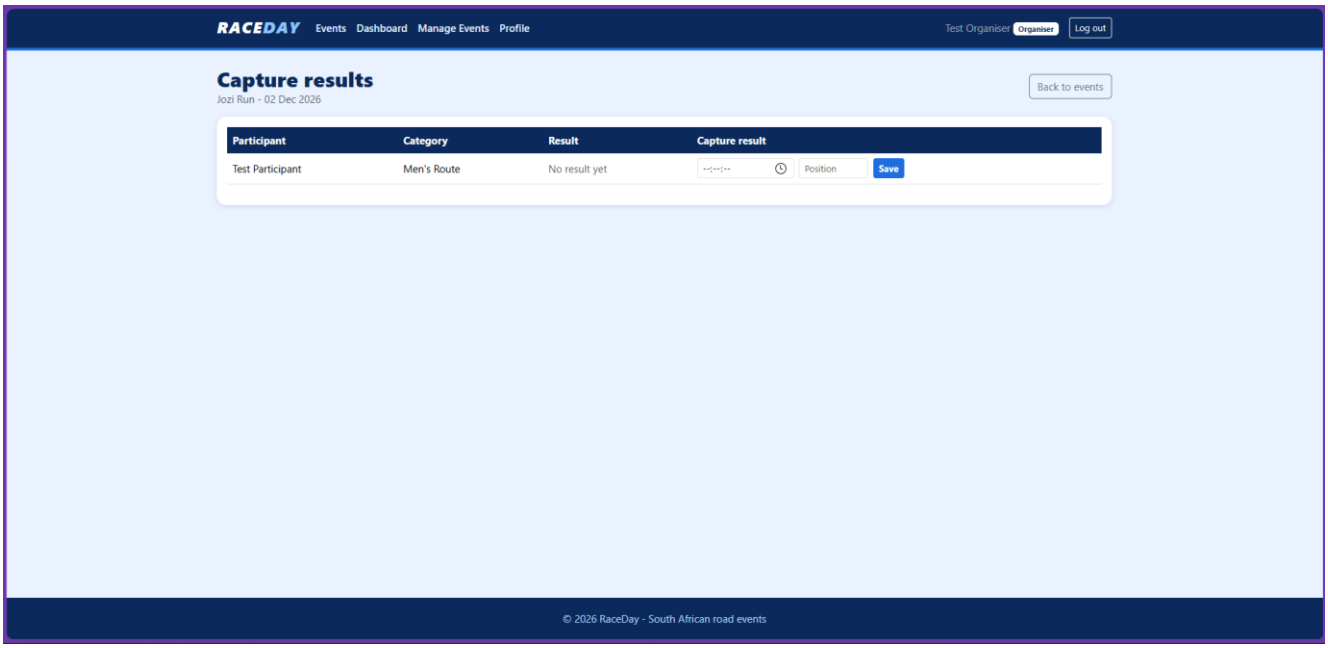



*Figure 10: Results capture page*

## 6. Participant Interface

- Home: all upcoming events with search, event type and upcoming or all filters.
- Event details: full event information and available categories.
- Enrol: a logged-in Participant chooses a category and enrols; the enrolment is saved immediately in the system with the status Pending.
- My Enrolments: every event the Participant entered, with the category and a colour-coded status (green for Confirmed, amber for Pending).
- My Results: event name, date, category, finish time and finishing position shown as, for example, 47th out of 312 finishers.
- Profile: view and update personal information and upload a profile picture.

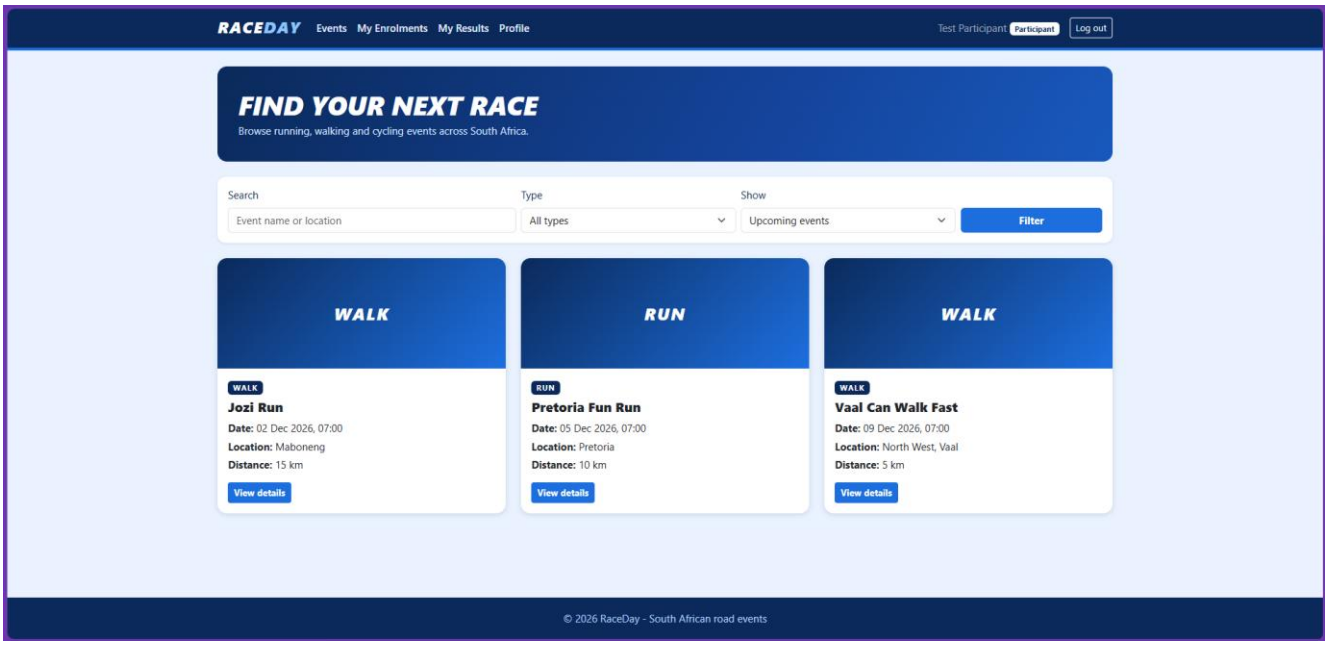



*Figure 11: Event details page with the enrol form (Participant)*

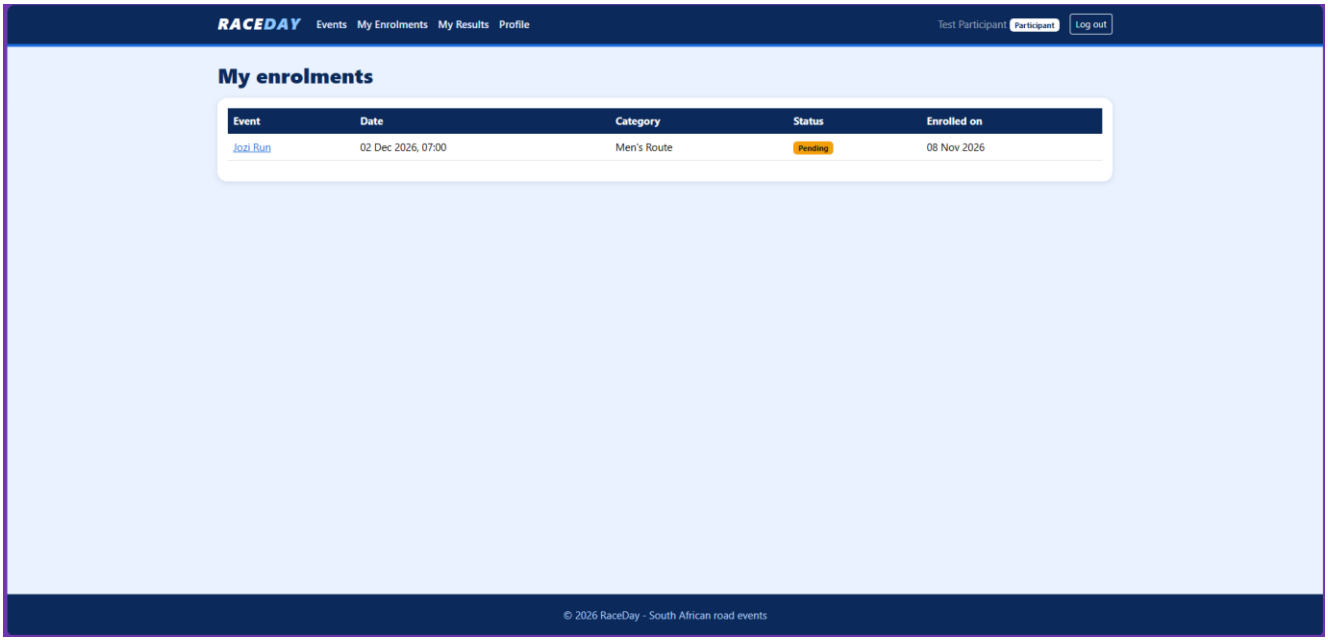



*Figure 12: My Enrolments page with colour-coded status*

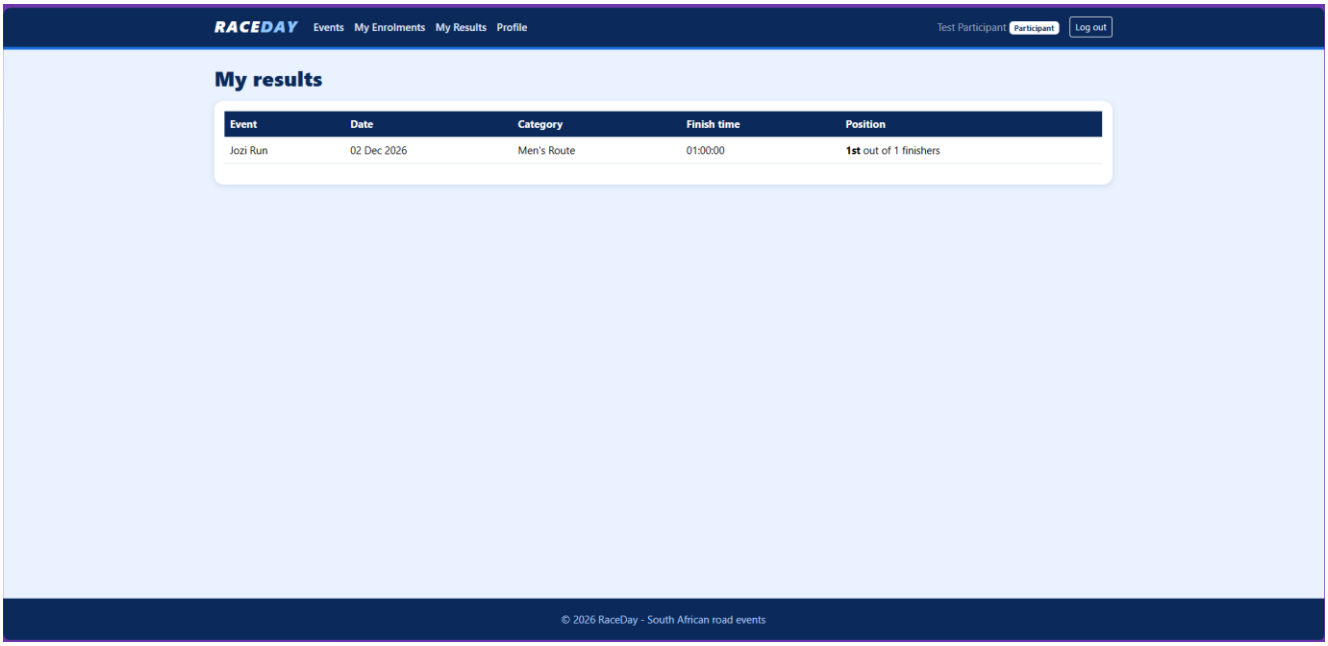



*Figure 13: My Results page*

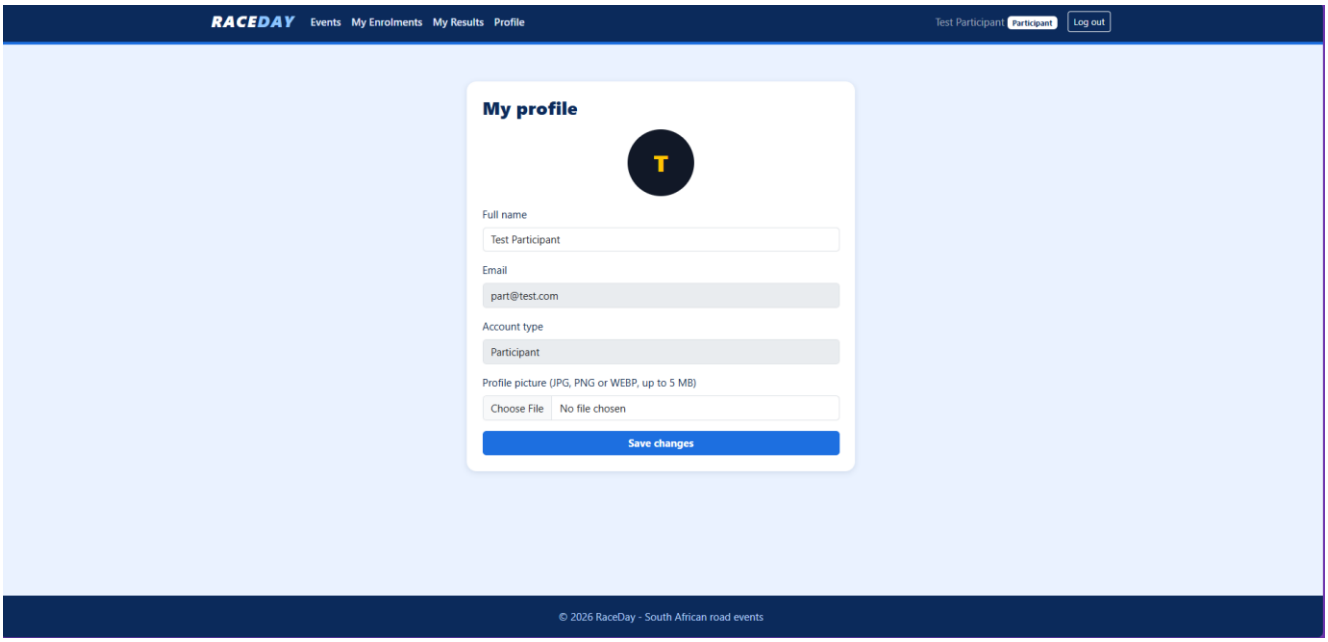



*Figure 14: Profile page*

## 7. Azure Blob Storage

Azure Blob Storage is integrated into the API, not the MVC application. The API exposes an upload endpoint (POST /api/uploads/{kind}) that accepts an image, validates it, and stores it in a blob container named raceday-images.

- Organisers can upload an event banner when creating or editing an event; any logged-in user can upload a profile picture.
- Only JPG, PNG and WEBP images up to 5 MB are accepted. Event banners can only be uploaded by Organisers.
- Each file gets a random name (event-banner/ or profile-picture/ folders), so uploads never overwrite each other.
- The stored image is served back through the API (GET /api/uploads/files/...), so the event details page and profile page display it without the browser or MVC app talking to Azure.
- The storage connection string is kept out of the source code (user secrets locally), and the MVC application only ever calls the API.
- If storage is unavailable, the user sees a friendly message instead of an error page.

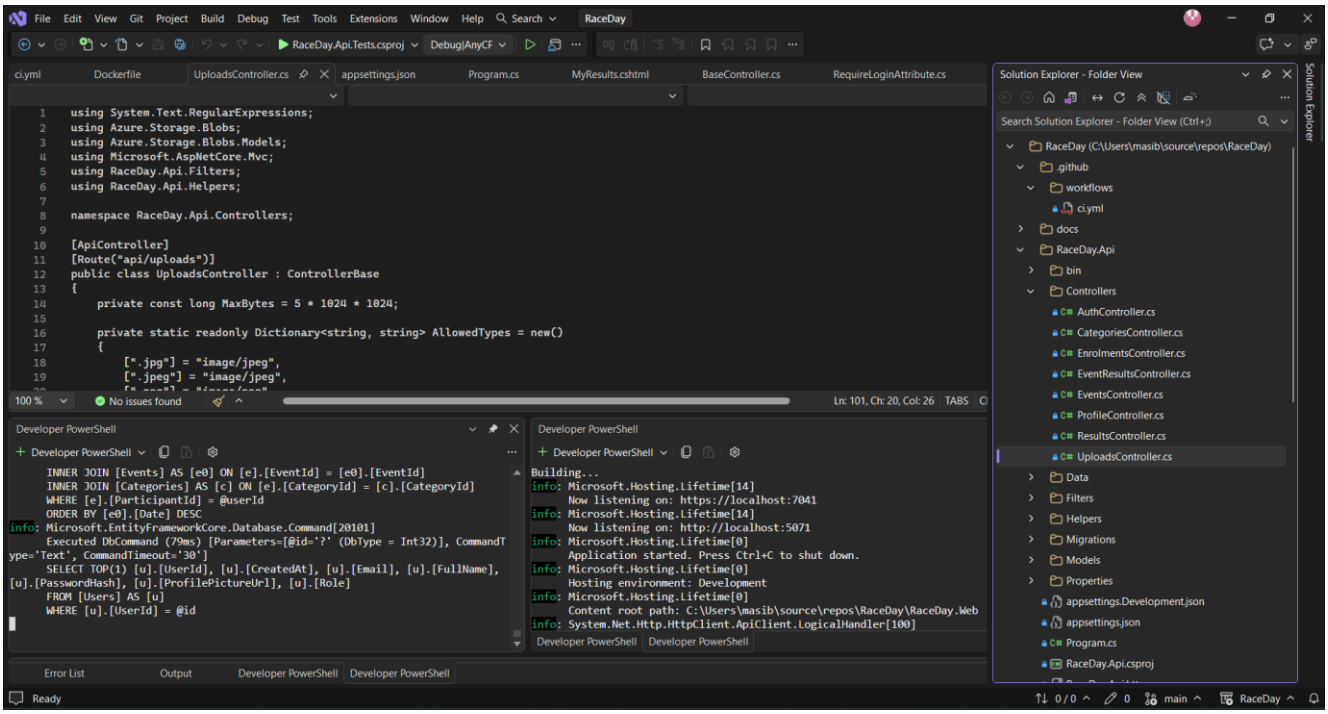



*Figure 15: Event details page showing an uploaded banner image*

## 8. Docker

The MVC application runs inside a Docker container built from a multi-stage Dockerfile: the .NET 10 SDK image restores and publishes the project, and the smaller ASP.NET runtime image runs the published output. The container listens on port 8080.

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY RaceDay.Web.csproj .
RUN dotnet restore
COPY . .
```

```
RUN dotnet publish -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "RaceDay.Web.dll"]
```

A docker-compose.yml file starts the container with a single command and tells the application where the API is through the ApiBaseUrl setting. The containerised application behaves the same as the local one.

```
docker compose up --build
```

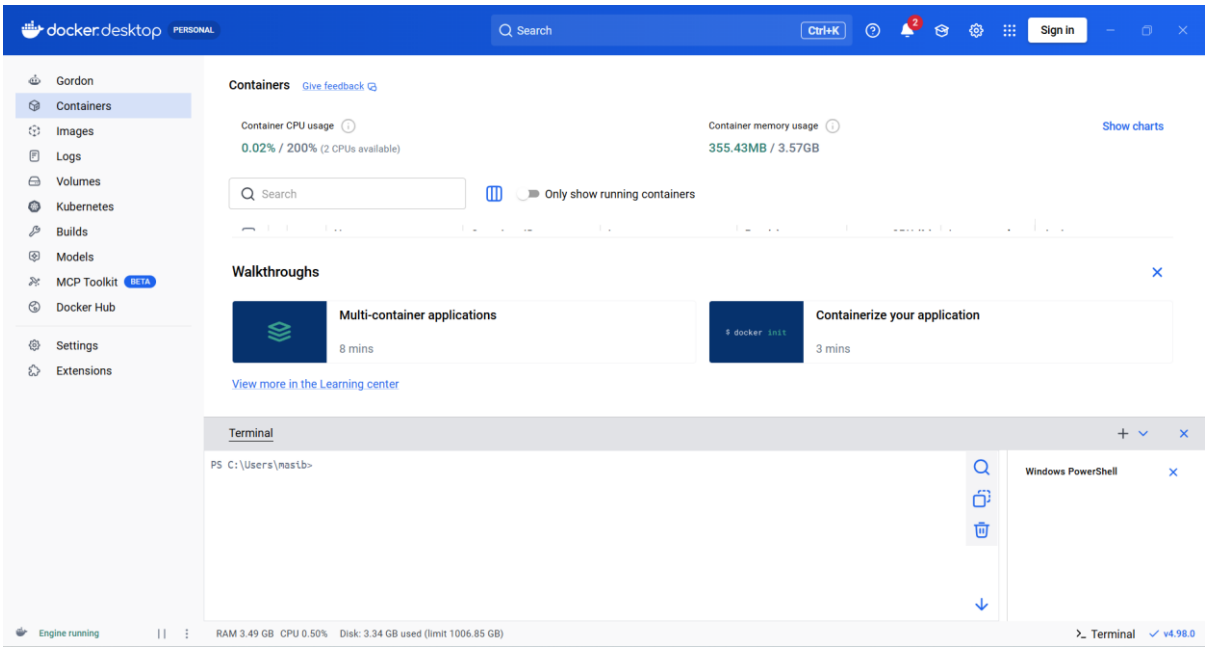



*Figure 16: Docker Desktop running (Engine running)*

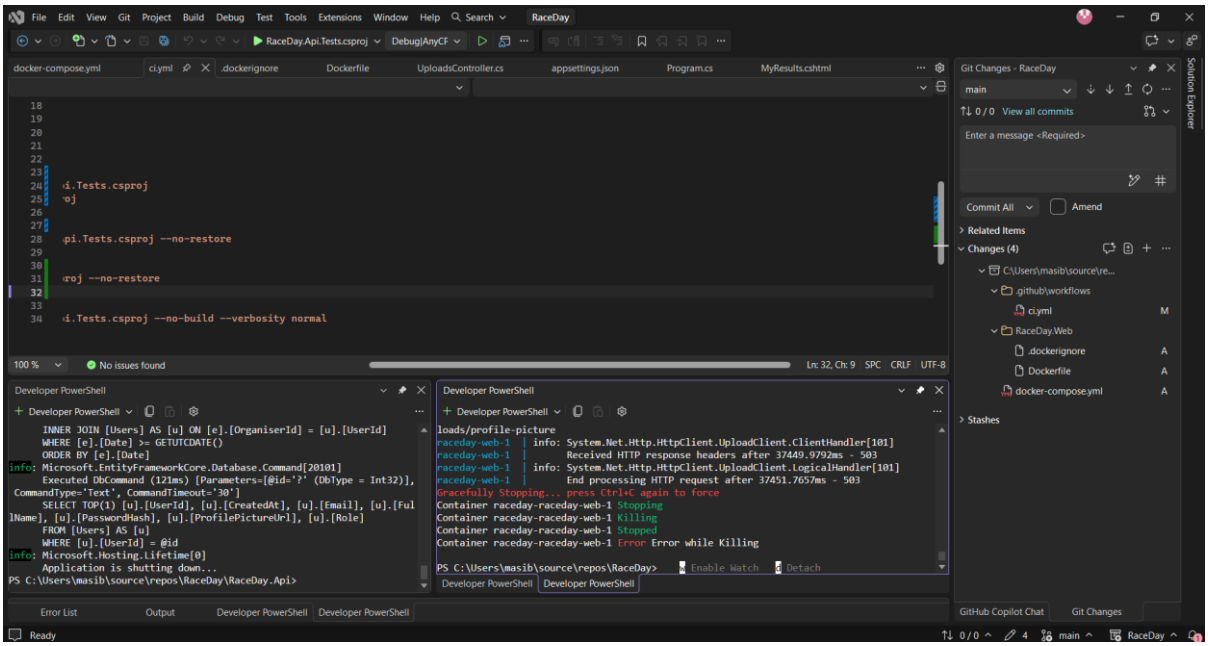



*Figure 17: The RaceDay web container started with docker compose (raceday-web-1)*

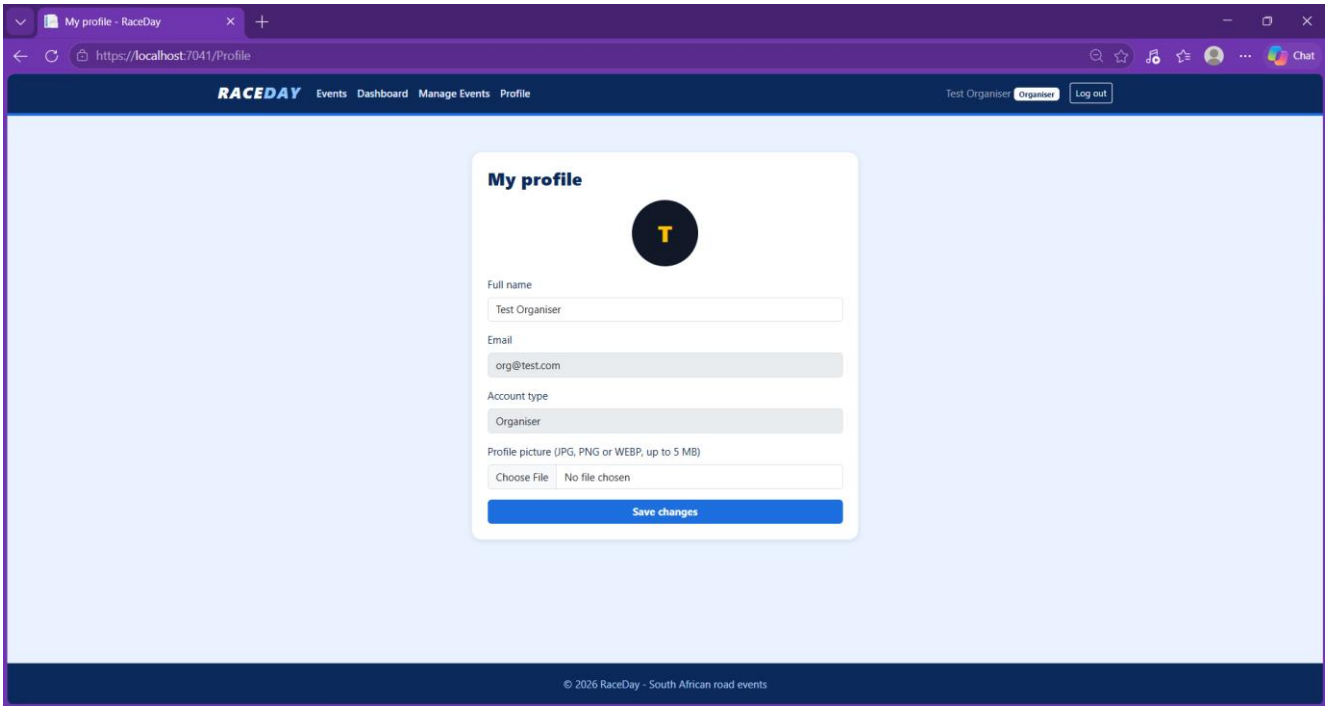



*Figure 18: The containerised application running at localhost:7041 (Profile page for the Organiser)*

## 9. User Interface Design

- Brand: a sporting, energetic theme built around dark navy, vivid orange and yellow, with bold italic headings and the RACEDAY logo in the navigation bar.
- Bootstrap 5 provides the responsive layout; one shared layout, one stylesheet and common card, table and button styles keep every page consistent.
- Role-appropriate navigation: the menu only shows the pages that the logged-in role may use.
- Enrolment statuses are visually distinct: green for Confirmed and amber for Pending.
- Invalid input is handled gracefully: forms validate on the client and the server and show clear messages, and API failures are shown as friendly alerts rather than raw errors.

## 10. GitHub and CI/CD

Part 3 work was pushed to the same repository with regular, meaningful commits. The GitHub Actions workflow was updated to also build the MVC project: it checks the docs folder, restores and builds the API, the tests and RaceDay.Web, and runs all unit tests on every push.

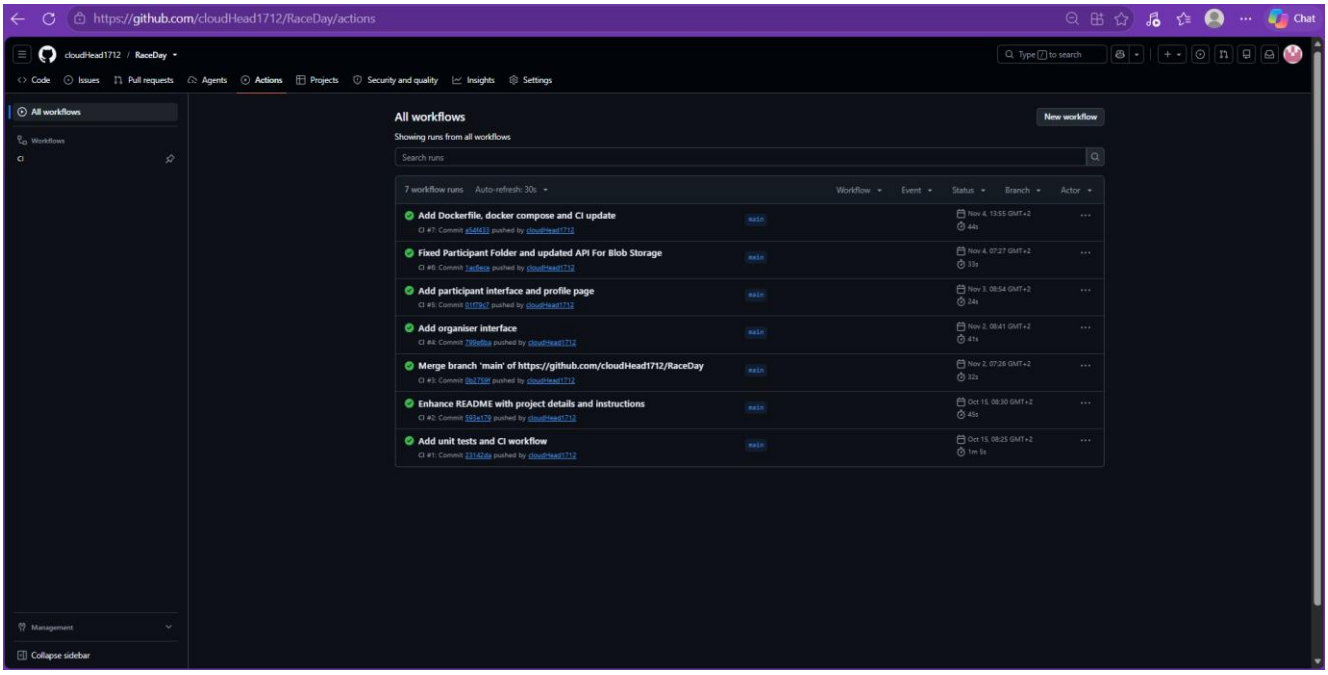



*Figure 19: Green GitHub Actions build for Part 3*

## 11. Setup and Run Instructions

- Install the .NET 10 SDK, Visual Studio 2026 (with SQL Server LocalDB) and Docker Desktop.
- Clone the repository.
- Start the API: cd RaceDay.Api, then dotnet run --launch-profile https.
- Start the web application: cd RaceDay.Web, then dotnet run --launch-profile https, and open the https address it prints.
- Docker: start the API with dotnet run --launch-profile http, then from the repository root run docker compose up --build and open http://localhost:8080.
- Blob Storage: set the connection string with dotnet user-secrets set "BlobStorage:ConnectionString" "<connection string>" in RaceDay.Api.

## 12. Deviations from the Plan

- Two endpoints were added for Part 3: GET /api/events/{eventId}/results (so the Organiser can see captured results) and POST /api/uploads/{kind} (Blob Storage upload).
- GET /api/results/mine now also returns the number of finishers per event, so the My Results page can show positions such as 47th out of 312.
- The event type filter uses the fixed seeded types (Run, Walk and Cycle).

## References


- Freeman, A., 2022. Pro ASP.NET Core 6: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Web API. 9th ed. New York: Apress.
- Microsoft, 2026. Overview of ASP.NET Core MVC. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview [Accessed 4 November 2026].
- Microsoft, 2026. Session and state management in ASP.NET Core. Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/app-state [Accessed 4 November 2026].
- Microsoft, 2026. Make HTTP requests using IHttpClientFactory in ASP.NET Core. Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/http-requests [Accessed 4 November 2026].
- Microsoft, 2026. Azure Blob Storage Documentation. Available at: https://learn.microsoft.com/en-us/azure/storage/blobs/ [Accessed 4 November 2026].
- Microsoft, 2026. Containerize a .NET app. Available at: https://learn.microsoft.com/en-us/dotnet/core/docker/build-container [Accessed 4 November 2026].
- Bootstrap, 2026. Bootstrap 5.3 Documentation. Available at: https://getbootstrap.com/docs/5.3/ [Accessed 4 November 2026].